PROJECT REVIEW II · CRYPTOGRAPHY & NETWORK SECURITY

Encrypted Cloud Vault

Searchable Symmetric Encryption over an Untrusted Cloud

Defense Notes & Anticipated Questions

Sowjhanya · Engineering, Year 4 · cyber_4year

Companion to the presentation deck and the live demo runbook

0

How to use this document

This document is the detailed backup for the slide presentation: full prose explanations, the exact measured numbers behind every chart, and a dedicated question bank at the end (Section 13) organised by topic. Read Sections 1–12 once before the review; keep Section 13 open on a second screen, or printed, during Q&A.

WHAT'S VERIFIED VS. WHAT NEEDS RECONFIRMING

Every architectural detail, code behaviour, and benchmark number below comes from reading the project's actual source files and its authoritative Windows benchmark data — nothing here is invented or extrapolated. The one exception is the live test-suite count and the live demo steps themselves, which must be re-run on your machine before presenting; this document says explicitly, each time, where that applies.

1

Motivation — why searchable encryption

Two conventional options both fail a cloud-storage vault. Encrypting everything with standard AES-GCM protects content perfectly, but the storage provider can no longer help you search — every query means downloading and decrypting the entire vault client-side. Storing an index of plaintext keywords keeps search fast, but now the provider (or anyone who breaches its disk) reads every document's real content and keywords outright.

Searchable Symmetric Encryption (SSE) is the middle path this project implements end to end: an encrypted index the untrusted server can run a matching loop over, without the server ever learning the keyword being searched, the filename, or the document content. This is the standard, practical scheme taught for this trade-off — more exotic alternatives (fully homomorphic encryption, ORAM) exist but carry order-of-magnitude higher computational or bandwidth cost, which is why HMAC-based static SSE is the appropriate choice at this scope.

2

Threat model & trust boundary

The system is built around exactly one trust boundary, and it is enforced in code, not only in documentation.

	Trusted client (client_app.py, :5000)	Untrusted cloud (cloud_app.py, :5001)
Holds	K_file, K_index — in one process-wide variable, in memory, for this session only	Nothing — zero key material ever reaches this process
Performs	Every encryption, decryption, and tokenization step	Storage (SQLite) and the HMAC search loop, over opaque labels it cannot interpret
Never	Talks to the cloud except in ciphertext / opaque tokens	Imports any of client/'s crypto code — enforced by an AST-based test, not convention

NAMED ASSUMPTION

The cloud is modelled as **honest-but-curious**: it follows the protocol correctly, but will look at anything it is handed. A cloud that actively lies about results, or silently drops them, is out of scope — tampering is at least *detected* (Section 8, D4), but not prevented outright. State this distinction plainly if asked; it is a standard, named limitation of this threat model, not a gap you discovered under pressure.

3

System architecture

The cloud and the client are two genuinely separate OS processes on separate ports, deliberately — Browser → `client_app.py` (:5000, trusted) → HTTP, base64 ciphertext → `cloud_app.py` (:5001, untrusted) → `vault.sqlite3` (documents / sse_index / meta tables). This makes the trust boundary a real process and network boundary that tests can inspect, not an informal convention living inside one script.

- **Stack:** Python 3, Flask, SQLite, `cryptography` (AES-256-GCM), `argon2-cffi` (Argon2id), HMAC-SHA256, `requests`, pytest.
- **Why SQLite, not Postgres/MySQL:** the cloud's storage layer (`cloud/storage.py`) is a thin, swappable persistence module; SQLite keeps the course project self-contained with zero external services.
- **Why Flask's dev server:** correct and sufficient for a single-operator local demo — named explicitly as a limitation (Section 11), not a production deployment choice.

4

Cryptographic design — key derivation

password → Argon2id(salt) → master key → HKDF-SHA256 → K_file / K_index. Argon2id is memory-hard by design: brute-forcing a stolen salt requires an attacker to pay real memory per guess, not just CPU time — something a fast hash (SHA-256) or a low-iteration PBKDF2 does not demand.

time=3, mem=64 MiB, par=4	57.9 ms	2 sub-keys
Argon2id cost parameters (OWASP baseline)	measured median unlock cost on the benchmark machine	K_file and K_index, from one master key via HKDF

DOMAIN SEPARATION

K_file = HKDF(master, info="vault:k_file:v1"); K_index = HKDF(master, info="vault:k_index:v1"). The versioned info labels mean a compromise of one sub-key's use never exposes the other — K_file only ever touches AES-GCM document/filename encryption; K_index only ever touches HMAC label/token derivation.

The salt itself is **not secret** — it is stored in the cloud's plaintext `meta` table by design; its only job is uniqueness per vault, not secrecy. A separate, non-secret `kdf_verifier` row (Section 8) proves the password is correct before any key reaches upload or index-rebuild logic.

5

Cryptographic design — document encryption (AES-256-GCM)

Both the file content and the filename are encrypted separately under K_file with AES-256-GCM. The cloud stores three columns per document: a 12-byte nonce, the ciphertext, and a 16-byte authentication tag. A tampered ciphertext or tag fails the authentication check loudly (HTTP 422) — it never silently returns corrupted plaintext.

2.7 GB/s	~900 MB/s	57.9 ms
peak median throughput, 256 KB payload (fits on-chip cache)	sustained at 16 MiB, once payload exceeds cache size	Argon2id unlock (repeated for reference)

Payload	Encrypt (MB/s)	Decrypt (MB/s)
1 KB	223.9	242.3
4 KB	743.1	770.5
16 KB	1,476.0	1,388.5
64 KB	2,206.6	2,365.9
256 KB	2,730.7	2,765.2
1 MB	854.6	823.6
4 MB	919.5	869.0
16 MB	893.5	963.5

Read this as: throughput peaks once a payload fits in on-chip CPU cache, then settles to a still-fast, memory-bound rate at larger sizes — a real hardware effect (AES-NI acceleration), not measurement noise. Either way, the cipher is never this system's bottleneck (see Section 11).

6

Searchable encryption — building the index

Tokenizer

- Lowercase; extract maximal [A-Za-z0-9]+ runs as tokens; drop tokens shorter than 2 characters.
- Drop a fixed English stop-word list; deduplicate keywords within one document.
- The exact same function runs at index time and at search time — by construction, the two cannot silently drift apart.

Static index (Πbas dictionary)

```
label = HMAC-SHA256(K_index, keyword || counter)
value = AES-GCM(K2, doc_id)   # K2 derived from K_index and the keyword
```

6,074	40,317	full rebuild
distinct keywords, 200-document benchmark corpus	(keyword, document) postings indexed	on every single upload — see why, below

WHY A FULL REBUILD, NOT AN APPEND

`build_index()` assigns each keyword's postings counters in doc_id-sorted order. Adding one new document can shift which counter an **existing** document receives for a keyword they both share — changing that document's label. An additive push would leave the old, now-orphaned rows behind. So every upload runs `clear_index()` followed by a full rebuild, inside one atomic `POST /index/replace` — a deliberate, documented trade-off whose cost is measured in Section 11, not an oversight.

7

Searchable encryption — the search protocol

keyword (browser) → tokenize/normalise → K1 = HMAC(K_index, '1' || keyword) → cloud's search loop (no decrypt) → K2 decrypts the returned doc_ids locally.

```
# cloud/cloud_app.py -- run_search_loop()
regenerate label = HMAC(k1, counter)
look up label in sse_index
stop at first miss, return every value found
```

- Only `k1` ever appears in the `/search` request body — there is no "query" or "keyword" field in its schema at all.
- The cloud's search route never calls decrypt on anything — it only recognises which opaque rows exist.

A DELIBERATE DEVIATION FROM THE TEXTBOOK

Section 06's pseudocode sends (K1, K2) together to the cloud. This implementation sends only K1 — K2 is derived locally, from the same inputs, exactly when it is needed to decrypt results. Since the cloud never touches decryption, K2 crossing the wire has zero functional benefit and only widens the key material ever exposed to the untrusted side. This is flagged directly in the code (`client/search.py`'s module docstring) as a one-line revert if literal fidelity to the pseudocode is preferred — a documented choice, not an unnoticed departure.

8

Security engineering — the H1 / H2 / F1 story

An independent audit ran 12 adversarial probes against the running system and found two High-severity findings. Both were closed with a minimal, justified fix; a follow-up audit then found one more gap in that fix, which was also closed. This progression is worth presenting as evidence of process, not as an admission of weakness.

Scenario	Before the fix	After the fix
Wrong password at unlock	200 “unlocked” — silent	401 “incorrect password”
Wrong password → upload	index destructively rewritten (H1)	structurally unreachable
`kdf_salt` **row deleted**	silently mints a new vault identity (H2)	409 — fails closed
Both KDF rows deleted (F1)	still bootstraps a new identity over real data	409, unless the vault is genuinely empty

The fix is one additional, non-secret meta row: `kdf_verifier` = AES-GCM-encrypt(K_index, a fixed constant). Every unlock must now decrypt it; failure means the password is wrong, and unlock refuses **before** any key reaches upload or index-rebuild logic. This adds no user accounts, sessions, or new key-derivation step — it reuses K_index, which Phase 1's unchanged unlock already produces.

230 tests passing at last verification (210 → 217 with the H1/H2 regression tests → 226 with the D1–D8 demonstration layer → 230 with the F1 follow-up + improved D3 coverage). **Reconfirm this number with a live** `pytest` **run before presenting** — do not quote it unverified (see Section 10).

9

Security demonstrations — D1 through D8

Every row below is generated automatically by `pytest_sessionfinish` from a real test run against the live two-process system (`tests/test_security_demonstrations.py`) — captured wire traffic and raw SQLite bytes, not hand-written claims.

#	What it demonstrates	Verdict
D1	Cloud never receives plaintext or keys	SECURE
D2	Deterministic tokens leak keyword repetition	LEAKAGE
D3a/b	Result-count & access-pattern leakage	LEAKAGE
D4	Tampering causes authenticated failure (422)	SECURE
D5	Missing/corrupt KDF metadata fails closed	MITIGATED
D6	Wrong password fails closed before any rebuild	MITIGATED
D7	Static rebuild cost is observable per upload	LEAKAGE
D8	A captured token still matches after later rebuilds	LEAKAGE

HOW TO FRAME THE LEAKAGE VERDICTS

“Secure” here means confidentiality of content, filenames, and keywords — not hiding search or access patterns, which is a known, textbook trade-off of essentially all practical searchable-encryption schemes, not a flaw specific to this implementation. Naming it explicitly, with evidence, is stronger than either hiding it or being surprised by the question.

10

Testing strategy & results

- **Unit:** `crypto.py, keymgmt.py, tokenizer.py, indexer.py, search.py` — pure functions, no network.
- **Integration:** `tests/conftest.py`'s `live_cloud` fixture runs the REAL `cloud_app.py` on a background thread with a real TCP port — not a mock.
- **Security regression:** the H1/H2/F1 attack sequences and the D1–D8 evidence tests, against that same live two-process topology.

Milestone	Passing tests
Phase 5 baseline	210
H1 / H2 fix	217
D1–D8 demonstration layer	226
F1 + improved D3	230

BEFORE PRESENTING

Run `python -m pytest` from the project root tonight, and use whatever number that run actually prints tomorrow — not the 230 above, unless this exact run reproduces it.

11

Performance evaluation

All figures below come from the project's existing, authoritative Windows benchmark run (`benchmarks/results/full/` and `full-b4/`) — nothing was re-run, re-measured, or extrapolated to produce these numbers.

11.1 Index rebuild cost

Checkpoint	Fetch docs (s)	Decrypt+tokenize (s)	Build index (s)	Replace on cloud (s)	Total (s)
10 docs	0.164	0.003	0.025	0.061	0.289
25 docs	0.501	0.005	0.057	0.096	0.606
50 docs	0.827	0.011	0.111	0.141	1.007
100 docs	1.612	0.022	0.208	0.305	2.710

HONEST LIMITATION, MEASURED NOT HIDDEN

All three authoritative attempts at a 200-document rebuild failed — not a crash, but the client's own 10-second HTTP request timeout being exceeded around document #80, exactly where this growth curve says it would happen. The fix is architectural (an incremental or forward-private index), not cryptographic: the cipher itself was never the bottleneck.

11.2 Search latency

Matching docs	Cloud-side loop (ms)	Client end-to-end (ms)
1	30.96	49.29
2	30.05	92.46
5	31.29	177.65
10	30.06	203.48
25	15.72	379.97
50	15.84	707.38
100	16.12	1,683.25

Cloud-side search time stays flat, 15–31 ms, regardless of match count. Client latency instead grows near-linearly, 49 ms to 1.68 s, because the client fetches every matching document individually to decrypt its filename — the bottleneck is that per-result fetch pattern, not the encrypted search loop.

11.3 Storage footprint (single 200-document snapshot)

10.63×	0.90 MB	9.58 MB
encrypted database file vs. original plaintext	plaintext content + filenames	final encrypted vault.sqlite3 file

Component	Size
Documents table (ciphertext + encrypted filenames)	0.92 MB
Encrypted search index (labels + values, 40,317 postings)	4.77 MB
SQLite structural overhead (pages, rowids, fragmentation)	3.89 MB

The encrypted search index alone is roughly half the total database — a real, necessary cost of letting the cloud search at all, not wasted overhead. Reported as a single measured snapshot (n=1, 200 documents), not a scaling law.

12

Known limitations & conclusion

Named in the code, exercised by a test, or measured on a benchmark above — not discovered by a reviewer first:

- No forward or backward privacy — a captured token still matches after later rebuilds (D8).
- Deterministic token linkability — repeat searches for the same keyword are visibly linkable (D2).
- Result-count & access-pattern leakage (D3a/b).
- Single-keyword search only — boolean/multi-keyword queries rejected by design.
- O(n) full index rebuild per upload — measured cost in Section 11.1.
- Single global in-memory session — one operator, one process, not multi-tenant.
- Flask's development server, not a production WSGI deployment.
- Argon2 cost parameters fixed, not persisted per-vault (open audit finding M1).

Future work

- An incremental or forward-private index, removing the O(n) rebuild and the D8 token-replay leakage together.
- Per-vault, persisted Argon2 parameters (closes audit finding M1).
- Session-scoped, multi-tenant key handling instead of one global in-memory KeySet.
- Production WSGI deployment in place of Flask's dev server.

CLOSING LINE FOR THE REVIEW

“Everything claimed in this project is backed by a test, a benchmark, or a piece of code that can be shown right now — including what does not yet work.”

13

Anticipated questions & answers

Organised by topic, matching Sections 1–12. Skim this once before the review; during Q&A, find the matching topic and answer in the same shape given here — name the mechanism, then the specific evidence (a test, a demo, a benchmark) that backs it.

Motivation & threat model

Q Isn't this just what HTTPS + a database already does?

A No — HTTPS protects the network hop; the threat model here is the storage provider itself (or anyone who steals its disk). The server is assumed honest-but-curious and must never see plaintext, keys, or keywords, even while it actively runs your search for you.

Q Why not homomorphic encryption or ORAM instead?

A Both exist and solve a broader problem, at order-of-magnitude higher computational or bandwidth cost. HMAC-based static SSE is the standard, practical middle ground for this scope — naming that trade-off explicitly is stronger than implying no alternative exists.

Q What if the cloud is actively malicious, not just curious?

A Out of scope by design. Tampering is at least detected — AES-GCM's auth tag fails loudly (D4) — but a malicious cloud silently dropping or altering results undetected is a named limitation of this threat model, not something claimed to be solved.

Cryptography

Q Why AES-GCM and not AES-CBC or ChaCha20-Poly1305?

A GCM gives authenticated encryption (confidentiality + integrity) in one primitive, with AES-NI hardware acceleration on virtually all modern CPUs. CBC alone has no integrity check and needs a separate MAC, which is easy to get wrong. ChaCha20-Poly1305 is a fine alternative where AES-NI isn't available — not a wrong choice to have made, just a different one.

Q How do you guarantee nonce reuse never happens?

A Each encryption draws a fresh 12-byte nonce from `os.urandom`. A 96-bit random nonce has a birthday-bound collision risk only after billions of encryptions under the same key — far beyond this project's scale. Name that as the actual, quantifiable residual risk, rather than claiming zero risk.

Q Why derive two separate keys instead of one key for everything?

A Key separation: K_file only ever touches AES-GCM document/filename encryption; K_index only ever touches HMAC label and token derivation. A bug or leak in one code path cannot cross into the other's key.

Q Is 57.9 ms unlock actually secure enough? Argon2id is supposed to be slow.

A It is tuned to the OWASP baseline, not maximised for slowness — real deployments size time_cost/memory_cost to the actual unlock hardware. The parameters are configurable (the `Argon2Params` dataclass); 57.9 ms is this benchmark machine's number, not a universal constant.

Q Where is the salt stored, and is that safe?

A The salt is not secret — stored in the cloud's plaintext meta table by design. Its only job is uniqueness per vault, not secrecy; the separate `kdf_verifier` row is what proves password correctness before any key is trusted.

Searchable encryption

Q Isn't sending K1 to the cloud itself a leak?

A K1 is single-keyword, single-purpose: it lets the cloud regenerate that one keyword's labels and nothing else. HMAC is one-way, so the cloud cannot invert K1 back to the keyword, or back to K_index.

Q Two searches for the same word — does the cloud notice?

A Yes, honestly — K1 is deterministic per keyword, so repeated searches produce a byte-identical token. That is exactly D2 in the security demonstrations: named and demonstrated, not hidden.

Q What happens if two documents share a keyword?

A Exactly the case the rebuild-not-append design exists to handle correctly — both documents get correctly-ordered postings under that keyword's label sequence after a full rebuild.

Q Doesn't a full-corpus rebuild leak the vault's size?

A Yes — total posting count and index row count are visible to the cloud. That is disclosed explicitly as D3a, a measured, known leakage, not swept under the rug.

Security engineering

Q Isn't adding a password verifier just adding authentication, which you said you wouldn't do?

A The verifier is a non-secret AES-GCM ciphertext of a fixed constant under the already-derived K_index; it adds no accounts, no sessions, and no new key-derivation step — it reuses key material Phase 1 already produced. This exact tension was raised and resolved deliberately, not smoothed over.

Q How do you know there isn't an H3 you haven't found?

A Don't overclaim: two independent audit passes found and closed 2 High findings plus 1 follow-up gap. 6 Medium findings (M1–M6) remain explicitly open and are named in Section 12, not hidden.

Q 230/230 — did you write tests after the fact just to inflate the number?

A No — each jump in the count is tied to a specific, named fix: 210 (functional baseline) → 217 (H1/H2 regression tests) → 226 (9 new D-series evidence tests) → 230 (F1 follow-up + improved D3 coverage). Reconfirm the live number before presenting; never quote a stale one.

Performance

Q So the project doesn't actually work past ~80 documents?

A Be precise: uploading and searching work fine past that. What fails is the FULL REBUILD every single upload triggers, once the corpus is large enough that fetching it all back exceeds a 10-second client timeout. Raising the timeout pushes the wall further out but doesn't fix the underlying O(n) growth — say that distinction explicitly.

Q Why not fix the rebuild cost before presenting?

A It's a deliberate, scoped limitation (the static-index decision), with the real cost measured and decomposed rather than guessed — more defensible than silently re-running until a lucky pass, or hiding the failure. The fix (an incremental/forward-private index) is named as future work.

Q Is 10.63× storage expansion acceptable?

A Name the components rather than only the ratio: roughly half is the encrypted search index itself (a real, necessary cost of searchability); the rest includes SQLite's own page/row overhead. It's a single measured snapshot (n=1 at 200 docs), not a scaling law — don't imply otherwise.

Q How would you fix the client-side search latency growth?

A Batch-fetch matched documents in one request instead of one GET per doc_id, or cache decrypted filenames — a concrete, scoped improvement to name if asked what comes next.

If asked to pick one weakness to fix first

A DEFENSIBLE ANSWER

The O(n) index rebuild — it is both the most measurable (Section 11.1's decomposition) and the most architecturally tractable (an incremental index is an engineering change, not a cryptographic redesign), and fixing it also closes the D8 forward-privacy gap at the same time.